

Appendix for EnTrust: Bringing Energy-Efficient Trustworthy Sensing for Battery-Free Sensor Nodes

This supplementary appendix provides the analytical, architectural, protocol, and evaluation details that complement the main manuscript. It first establishes the energy-harvesting, storage, intermittent-operation, task-execution, and communication-energy models for BFNs. It then details the sampling-free energy monitor, MDSM indicator processing and entropy-weight derivation, atomic cryptographic library, and DVFS/HEDA control logic. It also specifies the message flows, verification rules, assumptions, and security scope of the LMAP, LPUFA, and E²HA authentication protocols, and finally reports supplementary sensitivity and task-level evaluation results. Together, these materials provide the theoretical and implementation context needed to interpret the design and the evidence reported in the main manuscript.

A Extended Background and Mathematical Models

This section establishes the mathematical models and extended background for battery-free sensor nodes (BFNs), intermittent computing, energy-aware task scheduling, and energy-aware security primitives used by the EnTrust architecture in Section II of the main manuscript.

A.1 Battery-Free Sensor Nodes and Intermittent Computing

Battery-Free Sensor Nodes (BFNs) operate by harvesting ambient energy from sources such as radio frequency (RF), solar radiation, vibration, or thermal gradients. The harvested power $P_h(t)$ is inherently time-varying and can be modeled as a stochastic process:

$$P_h(t) = \eta \cdot P_{src}(t), \quad (1)$$

where η denotes the energy conversion efficiency and $P_{src}(t)$ represents the ambient energy source intensity.

Energy is temporarily stored in capacitors or supercapacitors. The stored energy $E(t)$ in a capacitor C with voltage $V(t)$ is given by:

$$E(t) = \frac{1}{2}CV^2(t). \quad (2)$$

A node becomes operational only when $E(t)$ exceeds a minimum operational threshold E_{on} :

$$E(t) \geq E_{on}. \quad (3)$$

BFNs follow a **charge-store-compute** execution model. During the charging phase, energy accumulates until the voltage reaches a turn-on threshold V_{on} ;

during computation, energy is consumed until it drops below a cutoff threshold V_{off} . The usable energy per cycle is thus:

$$E_{usable} = \frac{1}{2}C (V_{on}^2 - V_{off}^2). \quad (4)$$

Due to limited E_{usable} , tasks may not complete within a single cycle, leading to **intermittent computing**. If a task requires energy E_{task} :

$$E_{task} > E_{usable} \Rightarrow \text{task interruption occurs.} \quad (5)$$

To maintain forward progress, systems adopt checkpointing or idempotent execution models. The expected number of re-executions for a task can be approximated as:

$$N_{retry} \approx \left\lceil \frac{E_{task}}{E_{usable}} \right\rceil. \quad (6)$$

Here, N_{retry} denotes the approximate retry count in the intermittent execution model. This intermittent behavior introduces challenges in preserving execution state, ensuring data consistency, and minimizing redundant computation.

A.2 Energy-Aware Task Scheduling

In BFNs, task scheduling must explicitly incorporate energy dynamics. Let $E_{res}(t)$ denote the residual energy and E_i the energy required for task i . A basic feasibility condition for task execution is:

$$E_{res}(t) \geq E_i + E_{safe}, \quad (7)$$

where E_{safe} is a safety margin to prevent brownout during execution.

The scheduler aims to maximize system utility under energy constraints. This can be formulated as an optimization problem:

$$\max \sum_i U_i x_i \quad (8)$$

subject to:

$$\sum_i E_i x_i \leq E_{budget}, \quad x_i \in \{0, 1\}, \quad (9)$$

where U_i represents the utility (e.g., priority or importance) of task i , x_i is a binary variable indicating whether task i is selected for execution, and E_{budget} is the available energy within a scheduling window.

Dynamic Voltage and Frequency Scaling (DVFS) further enables fine-grained energy control. The dynamic power consumption is:

$$P_{dyn} = \alpha C_L V^2 f, \quad (10)$$

where α is the switching activity factor, C_L is the load capacitance, V is the supply voltage, and f is the clock frequency.

Execution time and energy consumption exhibit a trade-off:

$$E_{task} = P_{dyn} \cdot T_{exec}, \quad T_{exec} \propto \frac{1}{f}. \quad (11)$$

Here, T_{exec} denotes task execution time. Thus, reducing V and f lowers instantaneous power but may increase total energy due to prolonged execution under leakage, necessitating careful scheduling decisions.

A.3 Energy-Aware Security Primitives

Energy constraints also significantly impact security design, requiring primitives that are both lightweight and interruption-resilient.

Physically Unclonable Functions (PUFs) generate device-specific responses based on manufacturing variations. A PUF can be modeled as:

$$R = f_{PUF}(C, \delta), \quad (12)$$

where $f_{PUF}(\cdot)$ denotes the device-specific PUF mapping, C is the challenge input, and δ captures physical randomness. Due to environmental noise, repeated evaluations may produce slightly different responses:

$$R' = R \oplus n, \quad (13)$$

where n denotes noise. Reliability is typically quantified using intra-Hamming distance:

$$HD_{intra} = \mathbb{E}[d_H(R, R')], \quad (14)$$

where $d_H(\cdot, \cdot)$ is the Hamming-distance function and $\mathbb{E}[\cdot]$ denotes expectation over repeated evaluations, necessitating error correction mechanisms such as fuzzy extractors.

An **atomic lightweight cryptographic library** decomposes cryptographic operations into fine-grained atomic units. Let a cryptographic task consist of k atomic operations:

$$\mathcal{C} = \{op_1, op_2, \dots, op_k\}. \quad (15)$$

Here, k is the number of atomic operations and op_j denotes the j th operation in the cryptographic task. Each atomic operation op_j has bounded energy consumption:

$$E(op_j) \leq E_{atomic}^{\max}. \quad (16)$$

This ensures that:

$$E_{atomic}^{\max} < E_{usable}, \quad (17)$$

where $E_{atomic}^{\max}$ is the maximum energy budget of an individual atomic operation, allowing each operation to complete within a single energy cycle.

Security strength is modeled by the selected full-round security profile rather than by a reduced number of cipher rounds. Let $s \in \mathcal{S}$ denote a predefined profile

from the finite set $\mathcal{S} = \{S_1, \dots, S_8\}$, where each profile specifies a complete cipher path, authentication mode, tag length, freshness mechanism, and privacy floor. Its security capability is represented by

$$S = \sigma(s). \quad (18)$$

Here, $\sigma(\cdot)$ maps a profile identifier to its security capability. At runtime, MDSM selects the lowest-energy profile that meets both the required security floor and the available-energy constraint:

$$s^* = \arg \min_{s \in \mathcal{S}} E_{\text{crypto}}(s) \quad \text{subject to} \quad \sigma(s) \geq S_{\min}(t), E_{\text{crypto}}(s) \leq E_{\text{res}}(t). \quad (19)$$

In this formulation, s^* is the selected profile, $E_{\text{crypto}}(s)$ is the cryptographic energy required by profile s , $S_{\min}(t)$ is the time-dependent required security floor, and $E_{\text{res}}(t)$ is the residual node energy. If no full-round profile satisfies the required security floor, the node defers transmission or offloads the task rather than reducing cipher rounds. Thus, the atomic and reconfigurable design adapts energy cost through profile selection while preserving the standard full-round configuration of every adopted cipher.

B Ultra-Lightweight Atomic Encryption Generation

This section details the hardware and logical implementation of the adaptive encryption mechanism discussed in Section III of the main manuscript.

B.1 Sampling-Free Energy Level Monitor Circuit

BFNs face persistent energy scarcity because their power supply depends on intermittent and variable ambient sources such as light, RF, and vibration. Conventional energy-sensing methods rely on ADC sampling and MCU processing; even when a node is otherwise in a low-power sleep state, periodic wakeups for energy monitoring introduce additional overhead. To address this limitation, we design a micropower energy-level monitor circuit (Fig. 1) that continuously tracks the node's energy state without requiring clock-driven MCU or ADC operation. The circuit comprises two main components: an ultra-low-power amplifier (AMP) and an energy-level encoding module.

B.1.1 Ultra-low-power Amplifier

Since the voltage signal from the storage capacitor is often small in magnitude, direct sampling can suffer from low accuracy. The Charge Amplifier (CPA) first amplifies the input signal, with its gain determined by the input capacitance C_{in} and feedback capacitance C_f , described by:

$$V_{\text{out}} = \frac{Q_{\text{in}}}{C_f} = \frac{C_{\text{in}} \cdot V_{\text{in}}}{C_f} \quad (20)$$

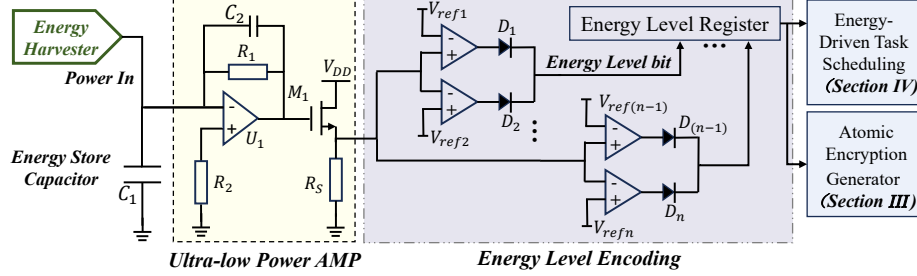


Figure 1: Sampling-free energy level monitor circuit.

Here, V_{out} and V_{in} are the output and input voltages, respectively; Q_{in} is the input charge; and C_{in} and C_f are the input and feedback capacitances. Proper selection of C_f and C_{in} enables tunable gain with minimal power overhead.

To ensure stable signal delivery to the comparator, the CPA output is buffered using a low-leakage CMOS voltage follower, which provides high input impedance and low output impedance, approximated by $V_{out} \approx V_{in}$.

B.1.2 Computation-Free Energy State Encoding

To address the challenge of accurately perceiving the energy state of terminal nodes under severe environmental energy fluctuations, which often cause failures in task scheduling and security-policy switching, this design introduces a window comparator array. The system configures multiple reference voltage thresholds $V_{ref1} \sim V_{ref4}$ on the integral output voltage $V_{out}(t)$ and continuously compares the real-time voltage levels. When $V_{out}(t)$ crosses a threshold, the corresponding comparator is triggered, resulting in a state transition at the output. This design enables high-resolution energy segmentation and accurate identification of the node's energy state, providing a reliable basis for subsequent scheduling and security decisions.

The proposed circuit precisely partitions the available energy range into five mutually exclusive levels and generates a unique encoding for each level, such as one-hot or binary representations. By leveraging this structured energy segmentation and flexible encoding control mechanism, the system is able to continuously monitor energy states and promptly react to energy variations, even when the MCU remains in a low-power sleep state.

With the proposed design, the measured energy state is converted into level codes for the control logic without periodic MCU/ADC sampling. These codes drive selection among predefined cryptographic profiles and scheduling actions. The corresponding power, latency, and decision results are reported for the evaluated implementation in the main manuscript.

Table 1: Key indicators and their classifications

Indicator	Description	Value Range and Explanation
Energy State (E)	Current available energy level of the node	E_1 (Very Low) to E_5 (Sufficient)
Task Load (L)	Complexity of the current processing task	L_1 (Low), L_2 (Medium), L_3 (High)
Data Privacy (P)	Sensitivity of task data	P_1 (Low), P_2 (Medium), P_3 (High)
Threat Level (T)	Detected security threat level in environment	T_1 (Low), T_2 (Medium), T_3 (High)
Node Role (R)	Functional importance of the node within the network	Controller > Relay > Sensor

B.2 Energy-Aware Security Decision (MDSM)

B.2.1 Input Indicator Normalization

The system selects five key input indicators, including energy state (E), task load (L), data privacy (P), threat level (T), and node role (R), which respectively characterize the node's residual energy, task complexity, data sensitivity, environmental threat intensity, and functional importance within the network. Since these indicators differ in physical meaning and scale, direct aggregation may lead to inconsistency. To ensure comparability and computational stability, all indicators are quantitatively modeled as continuous variables normalized to the range $[0, 1]$ based on measurable system parameters.

Specifically, all indicators are derived from observable system states, ensuring clarity and reproducibility. The energy state (E) is designed as a composite indicator that captures both residual energy and environmental energy availability through energy harvesting dynamics. Instead of explicitly modeling environmental factors such as solar intensity, RF energy density, or vibration frequency, these characteristics are implicitly reflected in the measured voltage level and charging behavior of the energy storage unit. Accordingly, the energy state is quantified as

$$E^* = \alpha \cdot \frac{V - V_{\min}}{V_{\max} - V_{\min}} + (1 - \alpha) \cdot \frac{\frac{\Delta V}{\Delta t}}{\left(\frac{\Delta V}{\Delta t}\right)_{\max}}, \quad (21)$$

where V denotes storage voltage, $V_{\min}$ and $V_{\max}$ are the lower and upper voltage bounds used for normalization, and $\Delta V/\Delta t$ represents the charging rate from an ultra-low-power energy detection circuit. The term $(\Delta V/\Delta t)_{\max}$ is the corresponding reference maximum charging rate. The parameter $\alpha \in [0, 1]$ balances residual energy and energy harvesting dynamics.

The task load (L) reflects both computational and communication demands, and is quantified using the estimated CPU cycles and data transmission volume:

$$L^* = \beta \cdot \frac{C}{C_{\max}} + (1 - \beta) \cdot \frac{D}{D_{\max}}, \quad (22)$$

where C and D denote the required computation and communication resources, respectively; $C_{\max}$ and $D_{\max}$ are their normalization reference maxima; and $\beta \in [0, 1]$ controls their relative importance.

The data privacy indicator (P) is defined based on the semantic sensitivity of the processed data and is mapped to a normalized value as $P^* = \frac{s(\text{type})-1}{2}$.

Here, $s(\text{type})$ represents the ordinal sensitivity level associated with the data type.

The threat level (T) is quantified by aggregating multiple observable security-related metrics, including abnormal communication frequency, authentication failure rate, and interference intensity:

$$T^* = w_1 \cdot \frac{f_a}{f_{\max}} + w_2 \cdot \frac{r_f}{r_{\max}} + w_3 \cdot \frac{I}{I_{\max}}, \quad (23)$$

where f_a , r_f , and I denote abnormal communication frequency, authentication failure rate, and interference intensity, respectively; the corresponding subscripted maxima are normalization reference values; and w_i are weighting coefficients satisfying $\sum_i w_i = 1$.

Finally, the node role (R) reflects the functional importance of a node in the network topology and is normalized as $R^* = \frac{R-1}{2}$. Through the above formulation, all indicators are represented as normalized continuous variables within a unified range, effectively eliminating dimensional heterogeneity and enabling stable and consistent weighted aggregation in subsequent decision-making processes.

B.2.2 Entropy Weight Method for Weight Calculation

To ensure objective, data-driven indicator weighting, the Entropy Weight Method (EWM) is adopted over subjective alternatives like the Analytic Hierarchy Process (AHP). Unlike AHP, which relies on expert judgment and risks bias, EWM derives indicator importance from intrinsic data statistics. This suits battery-free nodes (BFNs), where states such as energy availability and threat level fluctuate dynamically and unpredictably. By quantifying indicator dispersion, EWM assigns higher weights to those with greater variability, reflecting their higher information content and decision contribution.

The entropy values reported in Table 2 are computed from a comprehensive dataset encompassing 10,000 operational cycles of the prototype system. All cycles were obtained from real measurements on the implemented prototype rather than from simulation or synthetic data. The dataset was collected over a 30-day period under diverse energy harvesting conditions, including varying intensities of solar radiation (100–1000 W/m²), RF power (-20 to 0 dBm), and vibration sources. Each cycle records a normalized indicator vector (E, L, P, T, R), capturing dynamic variations in energy states, task characteristics, and environmental threats. The information entropy of each indicator is calculated from this historical matrix, where lower entropy indicates higher data dispersion. Based on these values, the information utility is derived and normalized to obtain the final weights.

To assess the stability of the derived weights, a sensitivity analysis was conducted. The dataset was partitioned into subsets based on specific energy sources (solar-only vs. RF-only) and varying sample sizes (2,000 to 10,000 cycles). As shown in Table 3, the resulting EWM weights exhibit a maximum fluctuation of less than 3.5% for the most influential indicator (E) across

Table 2: Entropy values and weights of indicators

Indicator	Entropy	Information Difference	Weight
E	0.8202	0.1798	0.3202
L	0.9128	0.0872	0.1554
P	0.8845	0.1155	0.2057
T	0.9059	0.0941	0.1676
R	0.9152	0.0848	0.1511

the evaluated partitions. This result indicates limited sensitivity to the tested energy-source and sample-size variations; it does not constitute a general statistical robustness guarantee beyond these evaluated conditions.

Table 3: Sensitivity Analysis of EWM Weights under Different Dataset Conditions

Dataset Condition	E	L	P	T	R
Full Dataset (10,000 cycles)	0.3202	0.1554	0.2057	0.1676	0.1511
Solar-only Environment	0.3145	0.1582	0.2014	0.1710	0.1549
RF-only Environment	0.3312	0.1510	0.2105	0.1623	0.1450
50% Random Sample (5,000 cycles)	0.3215	0.1548	0.2061	0.1665	0.1511

To evaluate the EWM-based allocation, it is compared with an Equal Weight (EQW) baseline, where all five indicators are assigned a fixed weight of 0.2. In the evaluated dataset, Table 4 shows that EWM yields fewer energy-depletion events and a higher task-success rate than EQW. By assigning larger weights to more informative indicators—including the energy state (E , 0.3202) and the data-privacy indicator (P , 0.2057)—EWM supports more risk-aware decisions. Because no inferential statistical test is reported for this comparison, these results are presented as empirical differences rather than statistically significant improvements.

Table 4: Performance Comparison: EWM vs. Equal Weight (EQW) Baseline

Metric	EQW (Equal Weight)	EWM (Proposed)	Improvement
Task Success Rate (%) $\uparrow$	78.5%	89.3%	+13.8%
Energy Depletion Rate (%) $\downarrow$	14.7%	4.2%	-71.4%
Average System Utility $\uparrow$	65.4	82.7	+26.5%
Dead Time Ratio (%) $\downarrow$	11.2%	3.1%	-72.3%

This methodology derives weights from the observed dataset rather than assigning them solely through expert judgment. Compared with *principal component analysis* (PCA), which primarily targets dimensionality reduction, EWM preserves a separate interpretable weight for each indicator. Its sensitivity to energy-source partitions and sample size is evaluated in Table 3.

Runtime implementation of entropy-weight updating: The entropy-

weight calculation is not executed during every security decision at the battery-free node. Instead, historical indicator samples are periodically aggregated and processed at the gateway, where the entropy values and the corresponding weight vector are recomputed at a relatively low frequency. The gateway then quantizes the five weights into a fixed-point representation and transmits the updated vector to the node.

During online operation, the node only reads the latest five weights from a local register and performs fixed-point multiply-accumulate operations to calculate the composite security score. Therefore, logarithmic operations, probability normalization, and historical-matrix storage are not required at the resource-constrained terminal. A weight update is applied only when the difference between the new and current weight vectors exceeds a predefined threshold, thereby avoiding unnecessary communication and configuration operations.

B.2.3 Composite Scoring Function

The final security scoring function adopts a weighted summation form:

$$\text{Score} = w_E E_{\text{norm}} + w_L L_{\text{norm}} + w_P P_{\text{norm}} + w_T T_{\text{norm}} + w_R R_{\text{norm}} \quad (24)$$

where E_{norm} , L_{norm} , P_{norm} , T_{norm} , and R_{norm} are the normalized energy, task-load, privacy, threat, and node-role indicators, respectively; w_E , w_L , w_P , w_T , and w_R are the entropy-weight-derived weights and $\sum_i w_i = 1$. These quantification rules ensure that all indicators are comparable in scale and physically interpretable, improving the transparency and reproducibility of the proposed mechanism.

To improve practical system adaptability, the continuous score output is mapped to one of eight predefined security profiles to facilitate simple runtime control. Although the continuous score in $[0, 1]$ accurately reflects risk, real terminal security mechanisms such as encryption strength and authentication complexity are typically implemented discretely. Thus, equal-width interval partitioning maps the score to one of eight predefined security profiles. This simplification reduces hardware complexity and maintains smooth policy transitions, improving system responsiveness and stability. The discretization scheme also offers extensibility for flexible granularity and threshold adjustment according to application scenarios, balancing security and resource constraints dynamically. The profile identifiers S1–S8 are implementation labels rather than a strict ranking of cryptographic strength.

B.3 Library of Encryption Algorithms

B.3.1 Atomic Operators and Formal Modeling

To enable formal, flexible, and runtime-reconfigurable construction, algorithms are represented as a library of reusable atomic operators:

$$A = \{f_1, f_2, f_3, f_4, f_5\}.$$

These operators correspond to elementary cryptographic primitives. The proposed operator library is sufficient for the lightweight block ciphers considered in this work. A new cipher can be instantiated without additional hardware only when its nonlinear transformation, diffusion layer, key-schedule operations, and control flow can be completely expressed by the existing operator set. Otherwise, the library can be extended by introducing additional atomic operators.

- **Nonlinear Mapping** $f_1(\cdot)$: Implements configurable S-Box-based nonlinear substitution for SPN ciphers such as PRESENT and GIFT. Alternative S-Box implementations (e.g., 8-bit) can be incorporated when required by the target cipher.
- **Linear Diffusion** $f_2(\cdot)$: Implements linear diffusion operations such as bit permutation, shift, or rotation used by lightweight block ciphers.
- **Modular Addition** $f_3(x, y)$: Performs modular addition

$$f_3(x, y) = (x + y) \bmod 2^n.$$

where n is the operand word length.

- **Bitwise XOR** $f_4(x, y)$: Performs
- $$f_4(x, y) = x \oplus y.$$
- **Cyclic Rotation** $f_5(x, r)$: Performs parameterized left or right cyclic rotation by r bits.

Algorithm execution is realized by chaining the required atomic operators according to the selected cipher specification. For illustration, an ARX-based round can be represented as

$$Y_k = f_5(f_4(f_3(X_{k-1}, K_k), K_k), r),$$

where k indexes the cipher round, X_{k-1} and Y_k denote the input and output round states, K_k is the round subkey, and r is the rotation distance. The above expression illustrates the composition of reusable operators rather than the exact round function of any specific cipher.

B.3.2 Hardware-Level Modular Implementation

At the hardware level, each atomic operator is mapped to a reusable IP core with a lightweight streaming interface, similar to AXI-Stream, where input data (`data_in`) and key (`key_in`) are provided alongside control signals such as `valid`, `ready`, and `start`, and the processed output is delivered through `data_out`. This standardized interface enables seamless pipelining, time-multiplexed execution, and cross-algorithm reuse. All modules are orchestrated by a centralized control unit, which selects among pre-instantiated operators through configuration registers, enable signals, and hardware multiplexers according to runtime energy availability and security requirements. Flash/ROM is used only to store constants, profile parameters, or lookup tables, rather than to load cryptographic hardware modules at runtime.

B.3.3 Dynamic Operator Composition and Scheduling

Combined with the micro-energy-aware security decision mechanism, the control logic selects the lowest-estimated-energy feasible profile after enforcing the task-specific security floor. Unlike static monolithic designs, this supports a controlled trade-off between cryptographic cost and declared protection requirements under the evaluated energy constraints.

B.3.4 Architectural Advantages

The proposed library follows a plug-and-play paradigm for algorithms that can be expressed using the instantiated operator set. In that case, adding a supported algorithm (e.g., SIMON or GIFT) requires only an FSM update to invoke the existing operators in a new sequence; an algorithm requiring an unavailable primitive would require a corresponding operator extension. Resource sharing across f_1 – f_5 reduces silicon area and static power, while parameterized modules support multiple algorithm configurations.

Table 5: Typical module sequences of lightweight encryption algorithms

Algorithm	Representative Atomic Modules	Rounds
PRESENT	S-Box $\rightarrow$ P-Box $\rightarrow$ XOR Round Key	31
GIFT	S-Box $\rightarrow$ P-Box $\rightarrow$ XOR Round Key	28
LED	S-Box $\rightarrow$ Shift/Rotate $\rightarrow$ XOR Round Key	48
SPECK	ROR $\rightarrow$ ADD $\rightarrow$ XOR Round Key	27
SIMON	AND $\rightarrow$ ROL $\rightarrow$ XOR Round Key	44
T-HIGHT	ADD $\rightarrow$ XOR $\rightarrow$ ROL	32

Table 5 illustrates how the implemented cipher paths are constructed from reusable atomic operators. The reported resource results quantify reuse for the evaluated operator set and ciphers; support for a new cipher requires that its round function and key schedule be expressible using that set or that the library be extended.

C Energy-Aware Task Scheduling and Energy Models

This section details the task models, comprehensive energy consumption models, and the low-level hardware implementation of the HEDA architecture, which serve as the basis for the energy-aware scheduling decisions in Section IV of the main manuscript.

C.1 Task Model

In this framework, application tasks are modeled as a directed acyclic graph (DAG) $G = (V, E)$, where V represents the set of subtasks and E denotes the data or control dependencies among them. For each subtask $v \in V$, the following attributes are defined:

- **Computation workload $c(v)$** : Represents the required CPU cycles for subtask v , reflecting its computational complexity.
- **Data size $d(v)$** : Denotes the amount of data to be processed or transmitted by the subtask.
- **Computational density $\rho(v)$** : Defined as $\rho(v) = \frac{c(v)}{d(v)}$, it characterizes the computation-to-communication ratio of a subtask. Higher values favor local execution, whereas lower values favor gateway offloading.
- **Deadline $t(v)$ and slack time $s(v)$** : The deadline $t(v)$ represents the latest allowable finish time of the subtask. The slack time, defined as $s(v) = t(v) - S(v) - T(v)$, represents the maximum tolerable delay for scheduling purposes. Here, $S(v)$ denotes the earliest start time of subtask v , determined by the completion of all its predecessor subtasks, and $T(v)$ represents the estimated execution and data transmission time of v . Slack time guides dynamic scheduling and priority assignment.
- **Priority $p(v)$** : Reflects the importance or urgency of the subtask, allowing trade-offs between real-time performance and energy efficiency.
- **Memory and I/O requirements $m(v)$** : Specifies the memory and I/O resources required by the subtask, which are critical for scheduling under resource-constrained conditions.

Transmission delay is explicitly considered in this model, including bandwidth-dependent transfer time and protocol overhead, which are incorporated into $T(v)$.

C.2 Comprehensive Energy Consumption Models

In energy-harvesting or battery-free terminal devices, the available energy supply is not only highly constrained but also exhibits strong temporal intermittency. The total energy dissipation of a battery-free node can be classified into two major functional domains: communication-related energy E_{comm} and on-device computation energy E_{local} .

$$E_{\text{total}} = E_{\text{comm}} + E_{\text{local}}. \quad (25)$$

C.2.1 Communication Energy Consumption Modeling

The total communication energy is decomposed according to:

$$E_{\text{comm}} = E_{\text{tx}} + E_{\text{rx}} + E_{\text{ctrl}} + E_{\text{sec}},$$

where E_{tx} , E_{rx} , E_{ctrl} , and E_{sec} denote transmission, reception, protocol-control, and security-processing energy, respectively.

Transmission Energy Consumption: The energy for transmitting N packets of payload size L is given by

$$E_{\text{tx}} = N \times \left((P_c + \eta \cdot P_{\text{te}}) \times \frac{L}{R} \right), \quad (26)$$

where N is the number of transmitted packets, L is the transmitted packet length in bits, R is the physical-layer data rate, P_c is circuit power, P_{te} is transmit power, and η is power-amplifier efficiency. The transmission duration $\frac{L}{R}$ encapsulates the impact of physical-layer rate R on temporal energy cost. The circuit power P_c contains a static baseline component P_{base} and a modulation-related dynamic component P_{mod} :

$$P_c = P_{\text{base}} + P_{\text{mod}}, \quad P_{\text{mod}} \propto R^\beta, \quad \beta \approx 1. \quad (27)$$

To maintain a target SNR at the receiver, the required transmission power must compensate for propagation loss:

$$P_{\text{te}} = \frac{\text{SNR} \cdot N_0 \cdot d^\alpha}{\eta \cdot h}. \quad (28)$$

Here, SNR is the target signal-to-noise ratio, N_0 is the receiver-noise reference term, d is link distance, α is the path-loss exponent, and h is the channel power gain.

Power amplifier efficiency follows a saturation response:

$$\eta = \eta_{\text{max}} \cdot (1 - e^{-\gamma P_{\text{te}}}). \quad (29)$$

The parameters η_{max} and γ denote the maximum amplifier efficiency and the saturation-response coefficient, respectively.

The payload size and data rate are computed as:

$$L = \left(\left\lceil \frac{\text{size}}{\text{MTU}} \right\rceil \cdot B + \text{size} \right) \cdot 8, \quad (30)$$

$$R = B \cdot \log_2 \left(1 + \frac{S}{N + N_{\text{interf}}} \right) \cdot C. \quad (31)$$

In the packet-length expression, size and MTU denote payload size and the maximum transmission unit, while B denotes per-packet overhead size. In the rate expression, B denotes link bandwidth; S , N , and N_{interf} denote signal,

noise, and interference power; and the final factor C is the implementation-dependent rate coefficient used by the model.

Reception Energy Consumption: The reception energy model is

$$E_{\text{rx}} = N \cdot \left(P_{\text{rx}} \cdot \frac{L}{R} \right) + P_{\text{idle}} \cdot T_{\text{idle}}. \quad (32)$$

Low-Power Listening (LPL) replaces continuous reception with periodic channel sampling:

$$E_{\text{rx}}^{\text{LPL}} = N \cdot \left(P_{\text{rx}} \frac{L}{R} \right) + (P_{\text{wake}} T_{\text{wake}} + P_{\text{sleep}} T_{\text{sleep}}) \frac{T_{\text{idle}}}{T_{\text{cycle}}}. \quad (33)$$

Here, P_{rx} and P_{idle} are receive and idle power, and T_{idle} is the idle-listening duration. For LPL, P_{wake} , P_{sleep} , T_{wake} , T_{sleep} , and T_{cycle} denote the power and duration of the wake and sleep states and the complete listening cycle, respectively.

Protocol Control Overhead: Control signaling includes handshake packets, channel probing, ACK/NACK exchange, and synchronization preambles:

$$E_{\text{ctrl}} = N_{\text{ctrl}} \left(\frac{P_{\text{tx}} C}{R} + \frac{P_{\text{rx}} C}{R} \right) + E_{\text{proc}} N. \quad (34)$$

In LPL-enabled nodes, additional wake-ups multiply this overhead:

$$E_{\text{ctrl}}^{\text{LPL}} = N_{\text{wake}} \left[N_{\text{ctrl}}^{\text{per}} \frac{(P_{\text{tx}} + P_{\text{rx}}) C}{R} + E_{\text{proc}} N_{\text{ctrl}}^{\text{per}} \right]. \quad (35)$$

In these control-overhead expressions, N_{ctrl} is the number of control packets, N_{wake} is the number of wake-up events, $N_{\text{ctrl}}^{\text{per}}$ is the number of control packets per wake-up event, P_{tx} and P_{rx} are transmit and receive power, C is the control-packet length, and E_{proc} is the processing energy per control operation.

Secure Transmission Overhead: Security operations introduce substantial computational energy:

$$E_{\text{sec}} = N \cdot (E_{\text{enc}} \cdot L + E_{\text{dec}} \cdot L + E_{\text{auth}}). \quad (36)$$

If the system supports configurable security levels l , the generalized model becomes

$$E_{\text{sec}} = N \cdot (E_{\text{enc}}(l) L + E_{\text{dec}}(l) L + E_{\text{auth}}(l)). \quad (37)$$

Here, E_{enc} and E_{dec} are the per-bit encryption and decryption energy terms, E_{auth} is the authentication energy per packet, and l indexes the configured security level.

C.2.2 Modeling of Local Processing Energy Consumption

Local energy consumption originates from processor switching activities, memory accesses, leakage currents, and operation of peripheral units.

System Load Power Consumption: The processor’s power consumption consists of three major components:

$$P_{\text{dyn,sys}}(f) = \alpha C_{\text{eq}} V_{\text{op}}^2 f, \quad (38)$$

$$P_{\text{ind}} = P_{\text{mem}} + P_{\text{IO}} + P_{\text{aux}}, \quad (39)$$

$$P_{\text{sta}} = V \cdot I_{\text{leak}}. \quad (40)$$

Here, α is the switching-activity factor, C_{eq} is the effective switched capacitance, V_{op} is the operating voltage, f is the clock frequency, P_{mem} , P_{IO} , and P_{aux} are memory, I/O, and auxiliary power, respectively, and V and I_{leak} are supply voltage and leakage current.

Task Execution Time: For parallelizable operations:

$$T = \max \left(\frac{C}{f_{\text{cpu}}}, \frac{D_{\text{tx}} + D_{\text{rx}}}{B}, \frac{S}{R_{\text{sto}}} \right), \quad (41)$$

while serial operations satisfy:

$$T = \frac{C}{f_{\text{cpu}}} + \frac{D_{\text{tx}} + D_{\text{rx}}}{B} + \frac{S}{R_{\text{sto}}}. \quad (42)$$

In the task-time expressions, C is the required computation cycles, f_{cpu} is processor frequency, D_{tx} and D_{rx} are the transmitted and received data volumes, B is link bandwidth, S is the storage-I/O volume, and R_{sto} is storage throughput.

C.3 Low-Level Hardware Implementation of the HEDA Architecture

This section provides the detailed hardware implementation of the Hardware-Embedded Dynamic Adjustment (HEDA) architecture mentioned in Section IV.B of the main manuscript. It details the internal logic of the DVFS Manager, Voltage Controller, and Frequency Controller.

C.3.1 DVFS Scheduling Controller Overview

For BFNs, DVFS-enabled components—such as processing cores and task schedulers—generate voltage-frequency requests based on real-time energy states and workload demands. The DVFS controller comprises a manager, a voltage controller, and a frequency controller, which jointly orchestrate voltage-frequency adjustments to adapt system operation to dynamically varying energy conditions.

Upon receiving a request, the Manager consults a software-configured VF lookup table to obtain the target voltage and frequency for the desired performance level. It compares the current and target voltages to decide whether scaling is required, then sends commands to the voltage and frequency controllers.

The Voltage Controller adjusts supply levels by programming an off-chip PMIC via I²C, while the Frequency Controller configures PLL clock sources to reach the target operating frequency. Together, these controls enable BFNs to autonomously adapt system operation under limited energy availability and dynamically varying workloads.

C.3.2 DVFS Manager — Unified Request Management Unit

In BFNs, the system core module (DVFS MCU) generates voltage-frequency scaling requests based on energy states and computational loads, specifying a target performance level. The DVFS Manager then consults a software-configured voltage-frequency lookup table (VF.LUT) to retrieve the corresponding parameters. It compares these with the current system state to determine if scaling is required, and if so, issues commands with target parameters to the voltage and frequency Controllers.

To ensure stable operation, a configurable delay mechanism enforces that frequency switching occurs only after the voltage adjustment has fully settled, thereby mitigating potential operational risks. The DVFS Manager integrates several key functions:

- **Performance Level Decoding:** Receives DVFS requests and target levels, looks up voltage and frequency from a table, and compares with the current voltage to decide increase, decrease, or hold.
- **Control State Machine:** Applies priority logic (voltage up before frequency up, frequency down before voltage down) for stable transitions, with delay states to await voltage or frequency stabilization after writing to the PMIC register.
- **Timing Control:** Uses the `delay_cycles` register and counters to manage delay intervals, where $\text{actual delay} = \text{delay_cycles} \times \text{clock period } T$. State transitions are gated by completion signals for safe operation.
- **Fault Handling:** Detects anomalies such as voltage adjustment failures or operational timeouts. In the case of voltage mismatches, the module sets `voltage_fault` and logs diagnostic information; for timeout events, a `timeout_cycles` counter is employed, and the system reverts to the idle state if the threshold is exceeded.
- **Asynchronous Interaction:** Ensures reliable communication across multiple clock domains by employing double flip-flop synchronizers. It manages critical signals, including `voltage_ack`, `voltage_fault`, and `freq_ack`, thereby preventing timing violations, metastability issues, and signal glitches.

C.3.3 Voltage Controller — Voltage Regulation Module

The voltage controller performs discrete voltage adjustments for BFNs. It receives requests and target values from the DVFS controller, programs the off-chip PMIC via I²C, and is managed by a state machine (Figure 2, Table 6). To ensure reliability, it supports masked writes and read-back verification, providing accurate transmission and error feedback to upper layers.

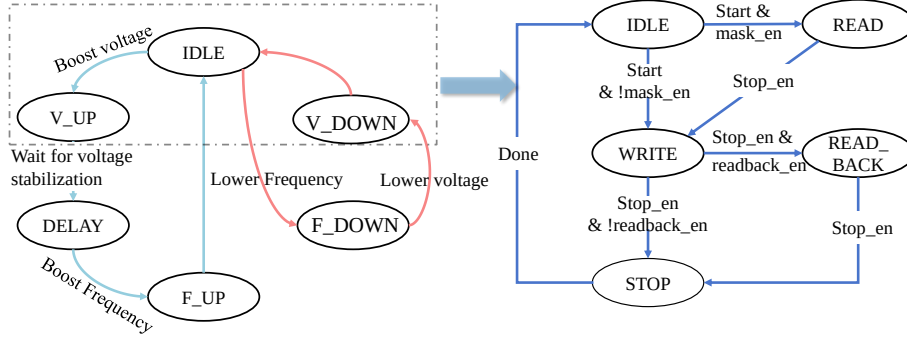


Figure 2: State machine of the voltage controller module, illustrating discrete voltage adjustment operations, communication with off-chip PMIC registers via I²C, and mechanisms for masked write and read-back verification.

Table 6: Descriptions of DVFS Control States

State Name	Function Description
IDLE	Idle state, waiting for a new DVFS adjustment request
VOLT_UP	Voltage increase phase, sending target voltage to the regulator module
DELAY_UP	Delay after voltage increase, waiting for voltage stabilization
FREQ_UP	Frequency increase phase, sending target frequency to the frequency module
FREQ_DOWN	Frequency decrease phase, reducing frequency as a priority
DELAY_DOWN	Delay after frequency decrease, waiting for frequency stabilization
VOLT_DOWN	Voltage decrease phase, lowering voltage accordingly

An APB interface is provided, allowing the CPU to write start, stop, and acknowledge commands for I²C timing control into internal registers; address and data registers are used for PMIC register mapping and voltage data transmission.

Masked Write Mechanism: To prevent unintended modifications to unrelated fields in multi-bit PMIC registers, the module supports an optional masked write mechanism. This technique first reads the current register value (*mdata*) and then computes the actual value to be written (*wdata*) using the following equation:

$$wdata = (mdata \wedge mask_value) \vee (data0 \wedge \neg mask_value) \quad (43)$$

Here, a mask bit of one preserves the corresponding bit of *mdata*, whereas a mask bit of zero updates that bit from *data0*. Finally, *wdata* is written back to the register. This method improves protocol compatibility and enhances portability.

Read-back Verification: After writing the target voltage value (*data0*) to the PMIC register, the module reads back the register content (*rdata*) and compares it with *data0*. A match confirms that the write operation has completed correctly, whereas a mismatch asserts an error signal (*err* = 1) to report a write failure.

Status Feedback: The error signal *err* is synchronized with corresponding acknowledge signals and determined by active functional configuration. The specific conditions under which the error signal is generated are summarized in Table 7.

Table 7: Voltage Controller Error Output Conditions

Functionality	Error Condition	Error Out-put
Basic write only	I ² C bus no response or write failure	1
Read-back verification enabled	$rdata \neq data0$	1
Masked write enabled	I ² C write phase failure	1
Masked write and read-back verification	$rdata \neq wdata$	1
Successful operation	Bus responds correctly, data matches	0

C.3.4 Frequency Controller — Clock Management Unit

The Frequency Controller enables dynamic performance adjustment for BFNs following energy restoration by supplying configurable clock signals to the DVFS MCU. It supports flexible clock source selection and single-stage frequency division, ensuring low-power, glitch-free transitions while maintaining overall system stability.

Upon receiving frequency configuration commands (**freq_cfg**) from the DVFS manager, the frequency controller selects one of five external PLLs via glitch-free multiplexing, disabling the current clock before enabling the target. A lightweight divider, controlled by **freq_cfg[15:8]**, supports 1/2, 1/4, and

1/8 ratios using three cascaded T flip-flops. All outputs pass through the glitch-free selector, ensuring stable and reliable clock management for BFNs.

D Energy-Aware Authentication Framework and Protocols

To address the heterogeneity of IoT devices in terms of energy availability, computational capability, and security requirements [1, 2], this paper proposes an energy-aware hierarchical authentication framework.

D.1 Framework Definition and Design Principles

The framework provides a unified abstraction that organizes heterogeneous authentication primitives, including lightweight hashing, multifactor authentication, and PUF-based mechanisms, according to device energy states and hardware capabilities. This abstraction enables flexible and scalable identity authentication for devices ranging from passive sensors to moderately resource-constrained nodes under ultra-low-power conditions.

Based on this framework, three representative authentication protocols are developed for different operating regimes:

- **Lightweight Multi-factor Authentication Protocol (LMAP):** Designed for ultra-low-power and intermittently active nodes, this protocol leverages symmetric key primitives to reduce computational and communication overhead.
- **Lightweight PUF-based Authentication Protocol (LPUFA):** This protocol combines PUF challenge-response authentication with lightweight error correction for hardware-constrained edge devices.
- **Enhanced Energy-Aware Hash Authentication Protocol (E²HA):** This protocol uses HMAC-based message authentication with symmetric pre-shared keys and adapts communication overhead according to real-time energy availability while preserving the required authentication security floor.

The proposed framework is characterized by three key aspects: energy awareness, capability matching, and security scalability. Protocol selection is aligned with device hardware features (e.g., PUF support and storage capacity), and the selected mode must satisfy the required authentication security floor. When the required full mode is infeasible, the device defers the task rather than weakening the cryptographic configuration.

Formally, the framework can be expressed as a mapping:

$$\mathcal{P} = f(E, C, S) \quad (44)$$

where $\mathcal{P}$ denotes the selected authentication protocol, $f(\cdot)$ is the protocol-selection mapping, E denotes the energy state, C represents device capability, and S indicates the required security level.

D.2 Lightweight Multi-factor Authentication Protocol (LMAP)

The lightweight multi-factor authentication protocol (LMAP) is designed for energy-constrained and intermittently powered sensor nodes, particularly battery-free nodes (BFNs) relying on ambient energy harvesting. To minimize communication and computational overhead, the protocol adopts a single-challenge single-response mechanism that combines cryptographic verification with auxiliary physical-layer anomaly checks.

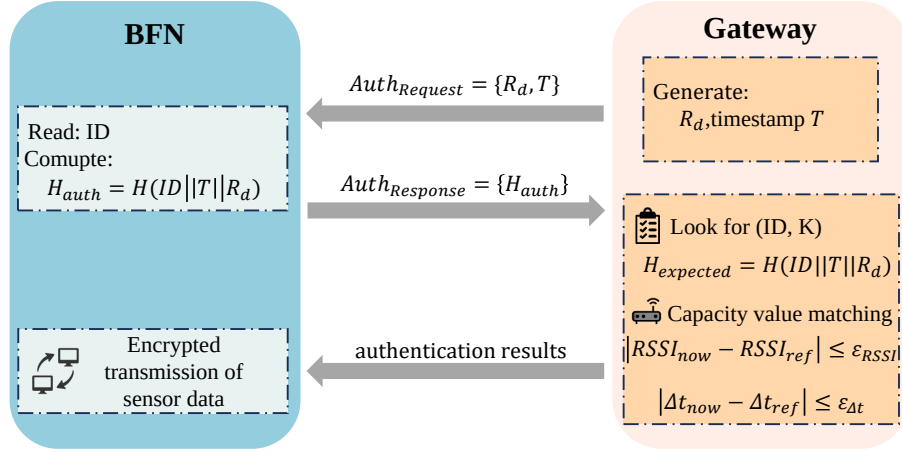


Figure 3: Overview of the LMAP authentication protocol for BFNs, showing the single broadcast-response interaction and lightweight security mechanisms.

D.2.1 Gateway Broadcasts Authentication Request

Due to the intermittent operation of BFNs caused by unstable energy availability (e.g., RF, light, or thermal sources), frequent communication should be avoided. Therefore, the gateway periodically or on-demand broadcasts a lightweight authentication request to trigger node responses without requiring continuous listening. The request is defined as

$$Auth_Request = \{R_d\}, \quad (45)$$

where R_d is a session-specific random challenge generated by the gateway. The gateway stores each outstanding challenge and marks it as consumed after successful authentication, thereby providing freshness without requiring a trusted clock at the battery-free node.

D.2.2 Terminal Computes Response and Sends Upstream

Upon receiving the authentication request, the terminal node immediately computes a lightweight authentication response using its locally stored identifier ID and secret key K . The authentication hash is generated as

$$H_{\text{auth}} = H(ID \parallel K \parallel R_d), \quad (46)$$

where $H(\cdot)$ denotes a hardware-friendly lightweight hash function. The terminal then transmits the response frame

$$\text{Auth_Response} = \{ID, R_d, H_{\text{auth}}\} \quad (47)$$

to the gateway. Including R_d lets the gateway identify the associated outstanding challenge without trial verification, while ID identifies the claimed device record.

D.2.3 Gateway Performs Multi-factor Authentication Check

Upon receiving the response, the gateway first checks whether the received R_d is an outstanding, unused challenge for the claimed device. It then retrieves the corresponding credential pair (ID, K) from the whitelist and recomputes the expected hash value as

$$H_{\text{expected}} = H(ID \parallel K \parallel R_d). \quad (48)$$

Authentication proceeds only if $H_{\text{expected}} = H_{\text{auth}}$. After acceptance, the gateway invalidates R_d so that a captured response cannot be reused.

To further enhance security, physical-layer features are incorporated as an additional authentication factor. Specifically, the gateway measures the received signal strength (RSSI) and response delay Δt_{now} , and compares them with the reference values RSSI_{ref} and Δt_{ref} , which are established through historical averaging over multiple successful authentications. The verification conditions are given by

$$\begin{aligned} |\text{RSSI}_{\text{now}} - \text{RSSI}_{\text{ref}}| &\leq \varepsilon_{\text{RSSI}} \\ |\Delta t_{\text{now}} - \Delta t_{\text{ref}}| &\leq \varepsilon_{\Delta t} \end{aligned} \quad (49)$$

where $\varepsilon_{\text{RSSI}}$ and $\varepsilon_{\Delta t}$ are tolerance thresholds determined based on statistical variations under normal operating conditions.

Authentication is considered successful only when both cryptographic verification and physical-layer constraints are simultaneously satisfied. Otherwise, the request is rejected, and the event can be logged for anomaly detection or subsequent re-authentication.

D.2.4 Security Analysis and Correctness

The correctness of LMAP follows from the deterministic consistency of the hash function, such that

$$H(ID \parallel K \parallel R_d)_{\text{device}} = H(ID \parallel K \parallel R_d)_{\text{gateway}}. \quad (50)$$

Replay resistance follows from the gateway-generated challenge R_d and the persistent outstanding/consumed-challenge state maintained at the gateway. The secret key K prevents an adversary without the credential from computing a valid response under the assumed hash security. RSSI and response delay are used only as auxiliary anomaly indicators because channel variation prevents them from serving as standalone cryptographic evidence. Consequently, LMAP combines a lightweight challenge–response exchange with optional physical-layer anomaly screening for resource-constrained, intermittently powered IoT environments.

D.3 Lightweight Physical Unclonable Function Authentication Protocol (LPUFA)

LPUFA uses the device-specific behavior of PUF circuits [3] for hardware-bound authentication on moderately energy-constrained devices. The protocol has registration and authentication phases; end devices do not store conventional cryptographic keys or perform encryption and decryption. Instead, identity verification uses enrolled challenge–response pairs (CRPs) together with lightweight error correction. Its security depends on secure enrollment, PUF unpredictability, response reliability, and protection of verifier-side records. The complete protocol flow is illustrated in Figure 4. The notation used in the following registration and authentication procedures is summarized in Table 8.

D.3.1 Offline Registration Phase

Before deployment, each terminal device must complete PUF modeling and identity binding within a secure offline environment. This phase is carried out collaboratively by the Verifier and the new Token, as described below:

- a) **ID Assignment and Initialization:** The Verifier generates a unique ID for each Token and writes it into the Token’s read-only memory.
- b) **CRP Collection:** The Verifier generates a pseudo-random set of m challenges, $\{C_1, \dots, C_m\}$, and applies them to the PUF circuit in the Token. Considering the potential impact of environmental factors such as voltage fluctuations and temperature variations, a *Jitter Measurement* mechanism is introduced to accurately characterize the stability of the PUF responses:
 - For each challenge C_i , the Verifier powers or resets the Token multiple times and collects the response $R_i^{(j)}$ for $j = 1, 2, \dots, K$, where K is the number of repeated samples collected for each challenge.
 - Each bit’s stability is computed, and a reference response R_i^{ref} is formed via majority voting.
 - Unstable bits are explicitly labeled as low-confidence indicators to facilitate efficient error correction during authentication procedures.

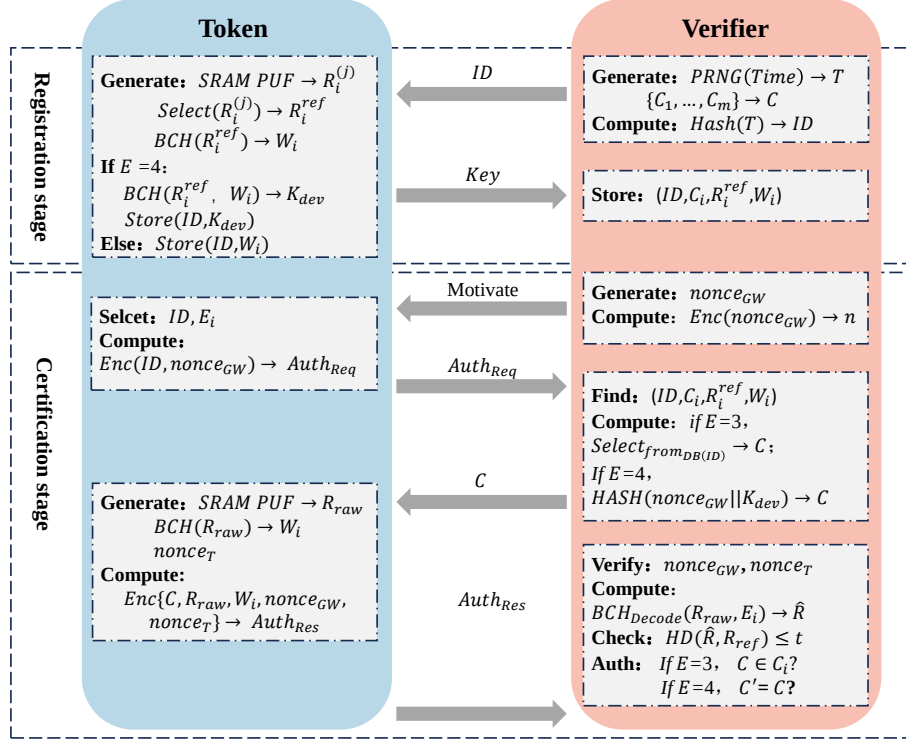


Figure 4: Protocol flow of Lightweight PUF Authentication (LPUFA).

- c) **Helper Code Generation:** Considering bit errors in practical deployments, each R_i^{ref} is encoded using BCH codes to generate helper data W_i , with decoding threshold t negotiated during system initialization.
- d) **Database Storage:** The Verifier securely stores each enrollment record in the database: $\langle ID, C_i, R_i^{ref}, W_i \rangle$. The Token does not store CRPs or helper data locally, which reduces terminal storage but does not by itself prevent every form of physical extraction.

After registration, the Token is deployed in the field. It does not store any cryptographic keys or CRPs, retaining only its device identifier and minimal lightweight hardware state. When external energy becomes available (e.g., via RF power), the Token is briefly powered on to compute real-time PUF responses, thereby minimizing both energy consumption and information leakage and making it particularly suitable for passive sensor node applications.

D.3.2 Interactive Authentication Phase

When a passive Token accesses the network or initiates task execution, it performs secure authentication under temporary external power, following a stan-

Table 8: Notation and Definitions in LPUFA Protocol

Symbol	Definition
Token	Passive device with embedded PUF; no long-term energy storage; operates only when externally powered.
Verifier	Authentication server with sufficient computation and storage resources.
ID	Unique identifier assigned to each Token by the Verifier.
C_i	The i -th challenge vector, randomly generated to stimulate the PUF.
$R_i^{(j)}$	Raw PUF response to C_i at the j -th sampling.
R_i^{ref}	Reference response for C_i , obtained via majority voting over multiple samplings.
C, R	Challenge and corresponding response of the PUF.
DB	Challenge-Response Pair (CRP) database storing $\langle ID, \{C_i, R_i\}_i \rangle$.
t	Error tolerance threshold to handle occasional jitter in PUF responses.
n	Number of challenges used per authentication session.

standard Challenge-Response protocol outlined below:

- a) **Auth Request:** On power-up, the Token reads its device identifier from non-volatile memory, generates a random session nonce nonce_T , and transmits the request:

$$\text{Auth.Request} = \{\text{ID}, \text{nonce}_T\} \quad (51)$$

- b) **Challenge:** Upon receiving the request, the Verifier retrieves the ID's record from the database, generates its own session nonce nonce_{GW} , and selects a challenge C_i along with its helper data W_i .

- *Level 3 (Standard):* The Verifier randomly selects an unused tuple $\langle C_i, W_i \rangle$ from the stored CRPs.
- *Level 4 (Dynamic):* The Verifier dynamically generates a new challenge C_i and predicts R_i^{ref} using a pre-trained strong-PUF machine-learning model.

The Verifier then sends the challenge message:

$$\text{Auth.Challenge} = \{C_i, W_i, \text{nonce}_{GW}\} \quad (52)$$

- c) **Response:** The Token applies the received challenge C_i to its PUF circuit to obtain the raw response R_{raw} . It then uses the received helper data W_i to correct noise-induced errors via lightweight BCH decoding, recovering the ideal response $R' = \text{BCH_Decode}(R_{raw}, W_i)$. To prevent exposing the PUF response in plaintext, the Token computes a cryptographic hash and sends the response:

$$V_T = \text{HASH}(R' \parallel \text{nonce}_{GW} \parallel \text{nonce}_T) \quad (53)$$

$$\text{Auth_Response} = \{V_T\} \quad (54)$$

- d) **Verification:** Upon receiving the response, the Verifier computes the expected hash value using its stored reference response R_i^{ref} :

$$V_V = \text{HASH}(R_i^{ref} \parallel \text{nonce}_{GW} \parallel \text{nonce}_T) \quad (55)$$

The Verifier checks whether $V_T \stackrel{?}{=} V_V$, where V_T and V_V are the Token-generated and Verifier-generated hash values, respectively. If they match, the Token is authenticated successfully.

This design decouples CRP collection from runtime authentication and shifts CRP storage and reference-response processing to the verifier. Under the stated enrollment, PUF-unpredictability, and helper-data assumptions, the nonce-bound response is intended to resist straightforward cloning and replay without storing a conventional secret key on the token.

D.3.3 Authentication Correctness

Authentication is accepted if and only if the following condition holds: $V_T = V_V$. Under the stated PUF and enrollment assumptions, this verification step provides:

- **Authenticity:** The response originates from the genuine PUF instance capable of reproducing R_{raw} close enough to R_i^{ref} .
- **Robustness:** Noise-induced errors in R_{raw} are tolerable and successfully corrected by W_i within the BCH threshold t .
- **Freshness:** Including both nonce_T and nonce_{GW} binds the response to the current session and prevents direct reuse of a previously accepted response.

D.4 Enhanced Energy-Aware Hash Authentication Protocol (E²HA)

With the rapid proliferation of IoT devices, BFNs face stringent energy constraints, particularly in passive or semi-passive scenarios with unstable power

and limited computing capability. Conventional authentication protocols impose heavy computation and long communication, leading to energy waste and frequent failures under fluctuating energy supply.

To address this, we propose E²HA, a lightweight HMAC-based authentication [4] protocol employing symmetric pre-shared keys for efficient identity binding and mutual authentication. The terminal energy level is dynamically incorporated to adapt authentication behavior and enforce security policies, enabling full authentication only when sufficient energy is available. The complete protocol flow is shown in Fig. 5, making E²HA well suited for low-power IoT deployments (e.g., wireless sensor networks and edge computing).

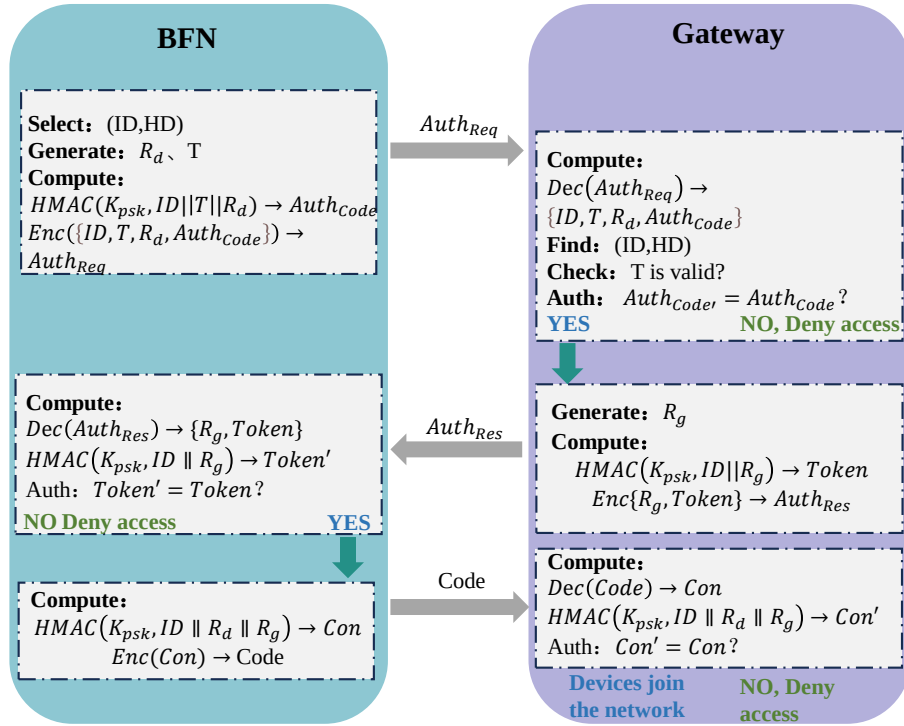


Figure 5: E²HA protocol flow, showing energy-aware mutual authentication between terminals and the gateway, with dynamic adaptation to terminal energy levels to reduce computation and communication overhead.

D.4.1 Initialization

The terminal device (Device) and the gateway (Gateway) share a unique device identifier ID and an HMAC pre-shared key K_{psk} (a fixed secret key) during the initialization phase.

D.4.2 Energy-Aware Authentication Policy

To explicitly incorporate energy awareness into the authentication process, the device dynamically selects its authentication mode based on its current energy level E .

$$\text{Mode} = \begin{cases} \text{Full Authentication,} & E \geq E_{\text{high}} \\ \text{Lightweight Authentication,} & E_{\text{low}} \leq E < E_{\text{high}} \\ \text{Deferred Authentication,} & E < E_{\text{low}} \end{cases} \quad (56)$$

Here, E_{high} and E_{low} are the upper and lower energy thresholds that delimit the three authentication modes.

Full authentication executes the complete three-way HMAC-based handshake, lightweight authentication reduces communication overhead by omitting the final confirmation from the device, and deferred authentication postpones the process until sufficient energy is available. This design aligns protocol execution with instantaneous energy availability, minimizing unnecessary failures and energy depletion.

D.4.3 Authentication Process

If $E < E_{\text{low}}$, the device enters the Deferred mode and remains in sleep state. Otherwise, the protocol proceeds as follows:

- a) **Device Sends Authentication Request:** The device generates a fresh random nonce R_d and transmits its identifier in plaintext to allow gateway lookup, accompanied by an HMAC for integrity and authenticity:

$$\text{Auth_Request} = \{ID, R_d, \text{Auth_Code}, \text{Mode}\} \quad (57)$$

where $\text{Auth_Code} = \text{HMAC}(K_{\text{psk}}, ID \parallel R_d \parallel \text{Mode})$.

- b) **Gateway Verifies Device Identity:** Upon receiving **Auth_Request**, the gateway first checks its persistent per-device replay cache and rejects any previously accepted nonce R_d . It then retrieves K_{psk} using the ID and computes:

$$\text{Auth_Code}' = \text{HMAC}(K_{\text{psk}}, ID \parallel R_d \parallel \text{Mode}) \quad (58)$$

If $\text{Auth_Code}' \neq \text{Auth_Code}$, the connection is rejected. After successful authentication, the gateway records R_d in the replay cache. This gateway-maintained state avoids relying on a trusted clock or persistent volatile state at the battery-free node.

- c) **Gateway Generates Challenge Response:** To demonstrate its authenticity to the device, the gateway generates a random nonce R_g and computes the HMAC token, binding it to the device's nonce R_d to ensure session freshness:

$$\text{Token} = \text{HMAC}(K_{\text{psk}}, ID \parallel R_d \parallel R_g \parallel \text{S2C}) \quad (59)$$

Here, the constant string **S2C** (Server-to-Client) acts as a directional tag for domain separation. It ensures that the gateway's token cannot be maliciously reflected back by an attacker as a valid device confirmation code. The gateway then sends $\text{Auth_Response} = \{R_g, \text{Token}\}$.

d) **Device Computes Token and Responds (Mode Dependent):** Upon receiving R_g , the device computes Token' locally. If it matches Token , the gateway is authenticated.

- If $\text{Mode} = \text{Lightweight}$, the device stops here to save transmission energy. The gateway has authenticated the device's request, and the device has authenticated the gateway's response; however, the gateway receives no explicit key-confirmation message from the device.
- If $\text{Mode} = \text{Full}$, the device replies with a confirmation message to explicitly finalize the handshake:

$$\text{Auth_Confirm} = \{\text{Confirm_Code}\} \quad (60)$$

where $\text{Confirm_Code} = \text{HMAC}(K_{\text{psk}}, ID \parallel R_d \parallel R_g \parallel \text{C2S})$. The **C2S** (Client-to-Server) tag guarantees cryptographic separation from the gateway's token.

e) **Gateway Verifies and Completes Authentication:** If the session is in Full mode, the gateway waits for Auth_Confirm and verifies:

$$\text{Confirm_Code}' = \text{HMAC}(K_{\text{psk}}, ID \parallel R_d \parallel R_g \parallel \text{C2S}) \quad (61)$$

If the codes match, the Full-mode session is complete. A Lightweight-mode session is accepted after the authenticated request and response, without this final confirmation.

D.4.4 Energy-Aware Verification Logic

Authentication success in E²HA is determined by cryptographic validity and the authenticated mode field. The device locally permits protocol execution only when $E \geq E_{\text{low}}$; the gateway does not independently measure the device's energy. Formally, the gateway accepts the device if and only if:

$$\begin{cases} \text{Auth_Code}' = \text{Auth_Code}, \\ R_d \notin C_{\text{replay}}, \\ \text{Confirm_Code}' = \text{Confirm_Code} \quad \text{if Mode} = \text{Full}, \end{cases} \quad (62)$$

and the device accepts the authentication if and only if:

$$\text{Token}' = \text{Token}. \quad (63)$$

Here, C_{replay} is the gateway-maintained set of previously accepted device nonces, and primed values denote the locally recomputed verification values.

This conditional verification separates the device’s local energy policy from the gateway’s cryptographic acceptance rule. Replay protection relies on fresh nonces R_d and R_g together with the gateway’s persistent per-device replay cache, while directional tags (S2C and C2S) provide domain separation against reflection. When $E_{\text{low}} \leq E < E_{\text{high}}$, the device omits **Confirm_Code**, reducing communication energy while retaining authenticated messages in both directions, but without explicit device-to-gateway key confirmation. The HMAC verification requirement is unchanged in both executing modes.

D.5 Theoretical Security Analysis of Authentication Protocols

This subsection provides a qualitative theoretical comparison of the three instantiated lightweight authentication protocols (LMAP, LPUFA, and E²HA) under the considered threat model. The analysis evaluates their resistance to common attack vectors in energy-constrained environments.

Table 9: Mechanism-level security scope of the lightweight authentication protocols

Protocol	Freshness / Replay Control	Authenticated Direction	Credential Basis	Payload Confidentiality	Principal Limitation
LMAP	Gateway-issued one-time challenge with persistent outstanding/consumed state	Device to gateway	Stored symmetric key; RSSI and delay are auxiliary anomaly indicators	No	Does not authenticate the gateway; stored-key extraction and relay attacks are outside the stated guarantees
LPUFA	Token and gateway nonces; unused CRP selection	Token to verifier	Enrolled PUF response and helper data	No	Depends on secure enrollment, PUF unpredictability/reliability, and protected verifier records; no verifier authentication
E ² HA	Device and gateway nonces with a persistent per-device replay cache	Both directions; explicit device key confirmation only in Full mode	Symmetric pre-shared HMAC key	No	Key extraction and physical capture are out of scope; Lightweight mode omits explicit final confirmation

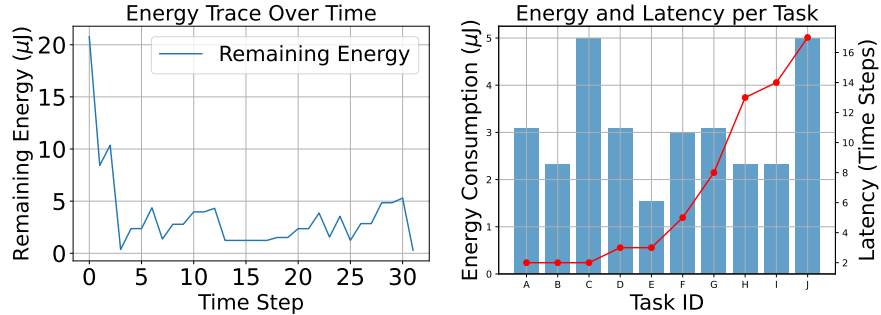


Figure 6: (Left) Residual energy fluctuation over time in the BFNs; (right) detailed view of energy oscillations at low energy levels.

Table 9 states the mechanism-level scope and assumptions of the three protocols without treating a binary check mark as empirical proof of attack resistance. None of the authentication protocols alone provides payload confidential-

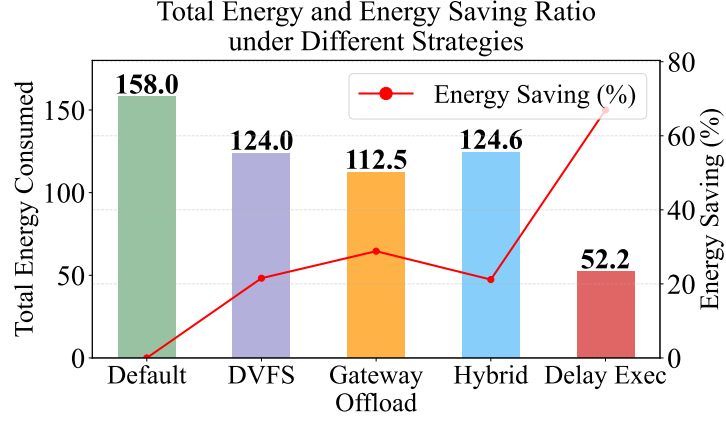


Figure 7: Total energy consumption with and without DVFS.

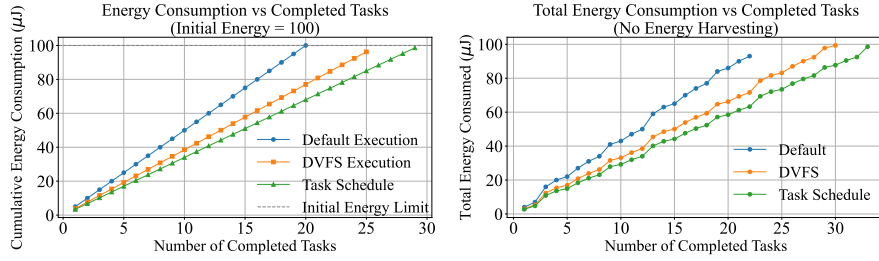


Figure 8: (Left) Comparison of cumulative energy consumption for different scheduling strategies without energy harvesting; (Right) Energy consumption improvement under energy harvesting conditions.

ity; protection of sensed payloads against eavesdropping requires the separate encryption layer described in the main manuscript.

LMAF provides device-to-gateway authentication using a one-time gateway challenge and a stored symmetric key, with RSSI and delay used only for anomaly screening. LPUFA provides hardware-bound token authentication under explicit enrollment and PUF assumptions. E²HA authenticates messages in both directions through nonce-bound directional HMAC tags, while only Full mode supplies an explicit final confirmation to the gateway. This comparison makes the authentication direction, credential assumptions, confidentiality boundary, and principal limitations explicit.

E Supplementary Evaluation Results

This section provides supplementary details of the proposed energy-driven task scheduling mechanism, including implementation examples, data analysis, and experimental results. The additional figures and explanations complement the main manuscript by clarifying design choices and reporting performance trade-offs under the evaluated energy conditions.

E.1 Task Performance Evaluation

This experiment examines temporal energy variation, task execution, and the energy consumption of different scheduling strategies along the local execution path. On the evaluated platform, residual energy decreases during intensive task execution and subsequently oscillates at a lower level as intermittent harvesting replenishes part of the consumed energy. Figure 6 presents these measured patterns.

Task-level analysis indicates that task complexity, data dependencies, and computational intensity affect energy consumption and latency in the evaluated workload. The task scheduling mechanism uses these task attributes when selecting an execution mode. Figure 7 compares total energy consumption with and without DVFS and shows lower energy consumption for high-energy tasks under the evaluated DVFS configuration. Figure 8 further contrasts the cumulative energy consumption of the evaluated scheduling strategies with and without energy harvesting, thereby illustrating the influence of energy replenishment on task-level energy use. Together, the reported results support the practical value of energy-aware scheduling in the tested battery-free environment while remaining limited to the evaluated platform and workload conditions.

Summary

Together, these results provide supplementary evidence for the behavior of the proposed mechanism on the evaluated platform and workload; they are not intended to establish performance beyond those conditions.

References

- [1] H. M. S. Badar, S. Qadri, S. Shamshad, M. F. Ayub, K. Mahmood, and N. Kumar, “An identity-based authentication protocol for smart grid environment using physical uncloneable function,” *IEEE Transactions on Smart Grid*, vol. 12, no. 5, pp. 4426–4434, 2021.
- [2] Y. Lei, L. Zeng, Y.-X. Li, and H. Qin, “A lightweight authentication protocol for uav networks based on security and computational resource optimization,” *IEEE Access*, vol. 9, pp. 769–785, 2021.
- [3] P. K. Sadhu and V. P. Yanambaka, “Mc-puf: A robust lightweight controlled physical unclonable function for resource-constrained environments,” in *IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2022, pp. 452–453.
- [4] C. E. Castellon, S. Roy, O. P. Kreidl, A. Dutta, and L. Boloni, “Towards an energy-efficient hash-based message authentication code (hmac),” in *IEEE International Green and Sustainable Computing Conference (IGSC)*, 2022, pp. 1–7.